PLIER ICI

13 Une requête dans une requête

Ça sert à quoi ? Quand il faut **d'abord calculer quelque chose** (une moyenne, un maximum…) avant de pouvoir choisir les lignes. On met ce calcul **entre parenthèses**. L'ordinateur calcule **d'abord la parenthèse**, puis la grande requête utilise son résultat.

```
SELECT name, price FROM Sephora
WHERE price > (SELECT AVG(price) FROM Sephora);
```

Comment il fait : 1) la parenthèse donne la moyenne = 38,57 · 2) la requête devient `WHERE price > 38.57` · 3) résultat : Soin A, Rouge B, Rouge C, Mascara B.

La parenthèse renvoie…	J'utilise	Exemple
une seule valeur (moyenne, max…)	`=` `>` `<`	`price = (SELECT MAX(price) FROM Sephora)` → le(s) plus cher(s), ex aequo compris
une liste (plusieurs marques…)	`IN`	`brand IN (SELECT brand FROM Sephora GROUP BY brand HAVING COUNT(*) >= 3)`

Astuce : répéter le filtre dans la parenthèse
« Le mascara le moins cher » : `WHERE category = 'Mascara' AND price = (SELECT MIN(price) FROM Sephora WHERE category = 'Mascara')`. Sans le 2e filtre, la parenthèse donne le prix le plus bas de **toute** la boutique, et on ne trouve aucun mascara.

Niveau + : NOT EXISTS = « il n'existe aucun… »

On donne deux **surnoms** à la même table (`s` et `p`). Pour chaque marque `s`, l'ordinateur cherche un produit `p` de la même marque en édition limitée. S'il n'en trouve **aucun**, il garde la marque.

```
SELECT DISTINCT s.brand FROM Sephora AS s
WHERE NOT EXISTS (SELECT 1 FROM Sephora AS p
                  WHERE p.brand = s.brand AND p.limited_edition = 1);
```

14 De la question au SQL

« affiche », « liste » · « seulement », « dont »	`SELECT` · `WHERE`
« entre … et … » · « contient » · « Dior ou NARS »	`BETWEEN` · `LIKE` · `IN`
« les différentes », « sans doublon »	`DISTINCT`
« combien » · « total » · « moyenne » · « le plus cher »	`COUNT` · `SUM` · `AVG` · `MAX`
« par marque », « pour chaque catégorie »	`GROUP BY`
« les marques qui ont au moins 5 produits »	`GROUP BY` + `HAVING`
« plus cher que la moyenne »	sous-requête
« le top 5 », « les 10 plus chers »	`ORDER BY … DESC LIMIT`
« inconnu », « non renseigné » · « aucun produit qui… »	`IS NULL` · `NOT EXISTS`

! Les 8 pièges, et pourquoi

`'Dior'` → `'Dior'`	les apostrophes penchées de Word ne sont pas comprises
`100 000` → `100000`	un espace coupe le nombre en deux
`name, price, FROM`	pas de virgule après la dernière colonne
`= NULL` → `IS NULL`	NULL n'est égal à rien, même pas à NULL
`WHERE COUNT(*)` → `HAVING`	au moment du WHERE, les groupes n'existent pas encore
AND + OR sans ( )	AND passe avant OR : le sens change
LIMIT sans ORDER BY	on garderait des lignes au hasard
`SELECT name … GROUP BY brand`	une marque a plusieurs noms : lequel afficher ?

LICENCE 1 · INFORMATIQUE · ÉVALUATION

SQL — la fiche mémo expliquée

Tout ce qu'il faut savoir pour l'évaluation, avec une explication simple et un exemple pour chaque notion.

xseorkly.github.io/sql1 — cours, exercices corrigés, évaluations

1 C'est quoi, une requête ?

Une base de données est **un grand tableau** : ici la table `Sephora`. **Une ligne = un produit**, **une colonne = une information** (nom, marque, prix…). Une **requête** est une question écrite en SQL ; l'ordinateur répond par un **petit tableau**, le résultat. Une requête `SELECT` **ne modifie jamais** la base : elle lit seulement.

2 La requête complète, ligne par ligne

```
SELECT   brand, COUNT(*)    -- 1. Qu'est-ce que j'affiche ?
FROM     Sephora            -- 2. Dans quel tableau ?
WHERE    price > 20         -- 3. Quels produits je garde ?
GROUP BY brand              -- 4. Je fais des tas par… ?
HAVING   COUNT(*) >= 2      -- 5. Quels tas je garde ?
ORDER BY brand              -- 6. Dans quel ordre ?
LIMIT    10;                -- 7. Combien de lignes au maximum ?
```

Seuls `SELECT` et `FROM` sont obligatoires. Les autres lignes s'ajoutent **seulement si la question le demande**, mais **toujours dans cet ordre** :

`SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT`

Pour s'en souvenir : « **S**ophie **F**ait **W**hisky, **G**uy **H**abite **O**slo, **L**ise ».

L'ordinateur, lui, lit dans un autre ordre

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`

Il prend le tableau, jette des lignes, fait des tas, jette des tas, puis affiche, trie et coupe. C'est pour ça que `WHERE` ne peut pas utiliser `COUNT` : les tas n'existent pas encore.

3 La méthode : 6 questions avant d'écrire

Ne commence pas par écrire du SQL : **réponds d'abord à ces questions**, chaque réponse donne une ligne.

1. **J'affiche quoi ?** → `SELECT`
2. **Dans quel tableau ?** → `FROM`
3. **Je garde certains produits ?** → `WHERE`
4. **« par… », « pour chaque » ?** → `GROUP BY`
5. **Je garde certains tas ?** → `HAVING`
6. **Trier ? Les premiers ?** → `ORDER BY`, `LIMIT`

Exemple : « Les marques qui ont plus de 100 produits, de la plus grande à la plus petite. »
1 → `brand, COUNT(*) AS nb` · 2 → `Sephora` · 3 → rien · 4 → `brand` · 5 → `COUNT(*) > 100` · 6 → `nb DESC`

4 Les règles d'écriture

Texte : entre apostrophes droites	✓ `'Dior'` ✗ `Dior` ✗ `‘Dior’`
Nombres : sans apostrophes, un point, pas d'espace	✓ `29.5` `100000` ✗ `29,5`
Majuscules : avec `=`, elles comptent dans le texte	`'Dior'` ≠ `'dior'`
Fin : un point-virgule ; aller à la ligne ne change rien	`… ;`

✓ Avant de rendre ma copie

- ☐ Chaque requête respecte l'ordre **S F W G H O L** et finit par `;`
- ☐ Le texte est écrit **exactement comme dans la base** : `'CLINIQUE'`, `'tarte'`, `'Perfume'`.
- ☐ J'affiche **seulement** les colonnes demandées. En « je joue l'ordinateur », j'ai pensé au **NULL** et à l'**ordre**.

Impression : A4 paysage, **recto-verso, retourner sur le bord court**, taille réelle (100 %). Plier au milieu : la page 1 devient la couverture.

PLIER ICI

PLIER ICI

Choisir les colonnes et les lignes

Les exemples utilisent cette petite table (table `Sephora`, 8 produits). Le prix de Soin B est inconnu : *NULL*.

Pour réviser
Pour chaque exemple, cache le résultat avec ta main et retrouve-le toi-même sur cette table, ligne par ligne. C'est exactement l'exercice « je joue l'ordinateur » de l'évaluation.

name	brand	category	price	love
Rouge A	Dior	Lipstick	30	120
Rouge B	Dior	Lipstick	50	200
Rouge C	Nova	Lipstick	50	80
Mascara A	Dior	Mascara	20	150
Mascara B	Nova	Mascara	40	90
Soin A	Nova	Skincare	60	70
Soin B	Luma	Skincare	*NULL*	30
Soin C	Luma	Skincare	20	40

5 SELECT et FROM : choisir les colonnes

`SELECT` = **les colonnes** que je veux voir, séparées par des virgules (pas de virgule après la dernière). `FROM` = **le tableau**. `*` = toutes les colonnes.

```
SELECT name, price FROM Sephora; → les 8 produits, mais seulement 2 colonnes (nom et prix).
```

6 WHERE : choisir les lignes

`WHERE` est un **filtre**, comme une passoire. L'ordinateur regarde **chaque ligne, une par une** : si la condition est **vraie**, il garde la ligne ; sinon, il la jette.

```
SELECT name FROM Sephora WHERE brand = 'Dior';
Résultat : Rouge A, Rouge B, Mascara A (Rouge C est Nova : jeté).
```

`=` égal · `<>` différent	`price <> 50` : tous sauf ceux à 50
`>` plus grand · `<` plus petit	`price > 50` : **50 n'est pas gardé**
`>=` plus grand **ou égal**	`price >= 50` : 50 est gardé (le trait = « ou égal »)

Deux conditions : AND (et) / OR (ou)

`AND` : les **deux** conditions doivent être vraies. `OR` : il suffit qu'**une** soit vraie.

```
WHERE brand = 'Dior' AND price > 25 → Rouge A, Rouge B (Mascara A est Dior mais coûte 20 : il en manque un).
```

Piège : AND passe avant OR → mettre des parenthèses
On veut « Dior ou Nova, à plus de 25 ». ✗ `brand = 'Dior' OR brand = 'Nova' AND price > 25` garde **tous** les Dior, même Mascara A à 20.
✓ `(brand = 'Dior' OR brand = 'Nova') AND price > 25`. **Règle : AND et OR ensemble = parenthèses.**

7 Trois raccourcis : BETWEEN, IN, LIKE

`price BETWEEN 20 AND 40`	entre 20 et 40, **bornes comprises** (= `>= 20 AND <= 40`)
`brand IN ('Dior', 'Nova')`	la marque est **dans la liste** (= Dior OR Nova)
`name LIKE '%mascara%'`	le nom **contient** « mascara » n'importe où
`name LIKE 'Mini%'` · `'%Set'`	**commence** par Mini · **finit** par Set

`%` = un joker : « n'importe quoi, même rien » · `_` = exactement une lettre. `=` cherche le mot exact, `LIKE` cherche un morceau (et ignore les majuscules dans SQLite).

8 Ranger le résultat : DISTINCT, ORDER BY, LIMIT

`SELECT DISTINCT brand`	enlève les répétitions : Dior, Nova, Luma (une fois chacune)
`ORDER BY price`	trie du plus petit au plus grand (ASC, par défaut)
`ORDER BY price DESC`	du plus grand au plus petit (**DESC** = **DESC**endre)
`ORDER BY price DESC, name`	si deux prix sont égaux, on range par nom
`ORDER BY price DESC LIMIT 3`	les 3 plus chers : Soin A, Rouge B, Rouge C

Toujours trier avant de limiter : comme un podium, on classe d'abord, puis on garde les 3 premiers. Sans ORDER BY, LIMIT prend des lignes au hasard.

Compter, calculer, grouper

9 Les fonctions de calcul

Ça sert à quoi ? À **résumer beaucoup de lignes en un seul nombre** : combien, le total, la moyenne… Sans GROUP BY, le résultat fait **une seule ligne**. Avec un WHERE, l'ordinateur **filtre d'abord, puis calcule** (ex. : `COUNT(*) … WHERE brand = 'Dior'` → 3).

J'écris	Ça calcule	Sur la petite table
`COUNT(*)`	le nombre de **lignes** (« combien ? »)	8
`COUNT(price)`	les lignes où le prix est **rempli**	7 (Soin B est NULL)
`COUNT(DISTINCT brand)`	le nombre de valeurs **différentes**	3 marques
`SUM(price)`	le **total**	270
`AVG(price)`	la **moyenne**	270 ÷ **7** = 38,57
`MIN(price)` · `MAX(price)`	le plus petit · le plus grand	20 · 60
`ROUND(AVG(price), 2)`	arrondit à 2 chiffres après la virgule	38.57
`COUNT(*) AS nombre`	donne un nom à la colonne du résultat	colonne « nombre »

10 NULL : la case vide

NULL veut dire « je ne sais pas ». Ce n'est **pas 0**. Pour le chercher : `price IS NULL` (ou `IS NOT NULL`), **jamais** `= NULL` (qui ne trouve rien).

Ce qu'il faut retenir
1. Une comparaison avec NULL n'est jamais vraie : Soin B n'est gardé ni par `price > 20`, ni par `price <= 20`.
2. Les calculs **ignorent** les NULL (sauf `COUNT(*)`) : la moyenne divise par 7 et pas par 8, comme un élève absent qu'on ne compte pas.
3. Dans la vraie base, `rating = 0` = « pas encore noté ».

11 GROUP BY : un calcul pour chaque groupe

Image : tu fais **des tas de bonbons par couleur**, puis tu comptes chaque tas. `GROUP BY` fait les tas ; les calculs se font **dans chaque tas** ; on obtient **une ligne par tas**.

```
SELECT category,
       COUNT(*) AS nombre
FROM Sephora
GROUP BY category;
```

Tas : Lipstick = Rouge A, B, C · Mascara = Mascara A, B · Skincare = Soin A, B, C
Résultat : Lipstick 3 · Mascara 2 · Skincare 3 (8 lignes → 3)

Règle d'or
Dans le SELECT, on met **seulement** la colonne du GROUP BY et des calculs. ✗ `SELECT name, COUNT(*) … GROUP BY category` : le tas Lipstick contient 3 noms, lequel afficher ?

12 HAVING : choisir les groupes

Deux filtres différents. `WHERE` trie **les produits**, **avant** de faire les tas. `HAVING` trie **les tas**, **après** les calculs.

```
SELECT brand, COUNT(*)
FROM Sephora
WHERE price >= 30
GROUP BY brand
HAVING COUNT(*) >= 2;
```

1. WHERE	reste Rouge A, B, C, Mascara B, Soin A
2. GROUP BY	Dior : 2 · Nova : 3 (Luma a disparu)
3. HAVING	Dior 2 ✓ · Nova 3 ✓

Le truc pour choisir
La condition parle **d'un produit** (son prix, sa marque) → `WHERE`. Elle parle **d'un groupe** (combien, moyenne du groupe) → `HAVING`. `WHERE COUNT(*) > 2` est une erreur.

« Je joue l'ordinateur » : dans cet ordre

1. **WHERE** : je barre les lignes qui ne passent pas (une ligne NULL est barrée).
2. **GROUP BY** : je fais les tas · **calculs** dans chaque tas · **HAVING** : je barre les tas.
3. **ORDER BY** puis **LIMIT**. J'écris **seulement** le résultat (noms, nombres), dans l'ordre.

PLIER ICI